

[Paper Review] BART: Denoising Sequence-to-Sequence Pre-training

25기 분석

서영우

0. Abstract

- sequence-to-sequence model을 **사전학습**시키기 위한 denoising autoencoder Bart를 제시
- text를 임의의 noise 함수로 corrupt하고, 모델이 기존 text로 복원하는 방식으로 학습
- 문장의 순서를 임의로 섞고, 임의의 텍스트를 단일 mask token으로 대체한 결과 효과적
- BART는 text 생성 뿐만 아니라 문맥 이해에도 뛰어난 성능을 보임
- 어느 요인이 end-task 성능 향상에 가장 큰 영향을 미치는지 ablation study 진행

*ablation study : 요소를 제거하거나 변경한 후 성능 변화를 비교하는 실험 (제거 연구)

1. Introduction

- Self-supervised 기반 사전 학습 모델들이 큰 성공을 거두었으나, 각 모델들은 특정 타입의 end-task에 집중하여 유연하고 넓은 범위로 사용하는 데에 어려움을 겪음.
- BERT의 경우 문장에 마스크를 씌우고, 양방향 문맥을 모두 참고해서 마스크 속 단어를 맞추는 식으로 학습하므로 문맥 **이해** 능력은 뛰어났으나, 입력과 전혀 다른 새로운 문장을 **생성**하는 데 어려움
- GPT의 경우 Auto-regressive 디코더를 사용해서 text **생성** 능력은 뛰어났으나, 이전 단어 들만 보고 다음 단어를 예측하는, 단방향 방식으로 학습하여 문장 전체의 양방향 문맥을 **이해**해야 하는 문제에는 취약함
- BART의 경우 이러한 문제점을 해결하기 위해 **인코더-디코더** 구조를 채택함.
- 임의의 마스킹된 문장을 인코더가 받았을 때, 망가진 텍스트를 양방향으로 훑어보고 의미적인 내용을 encoding 진행한 후, 디코더에서 원본 텍스트의 시작을 받고 이전에 압축(encoding)한 의미를 계속 참고하면서 한 단어씩 순서대로 생성(decoding)하는 구조.
- BERT처럼 단순히 빈칸만 채우는 것이 아니므로, 어떤 임의의 noise function이 적용 가능해짐. 즉, **noising flexibility**의 장점을 가진다.

2. Related Works

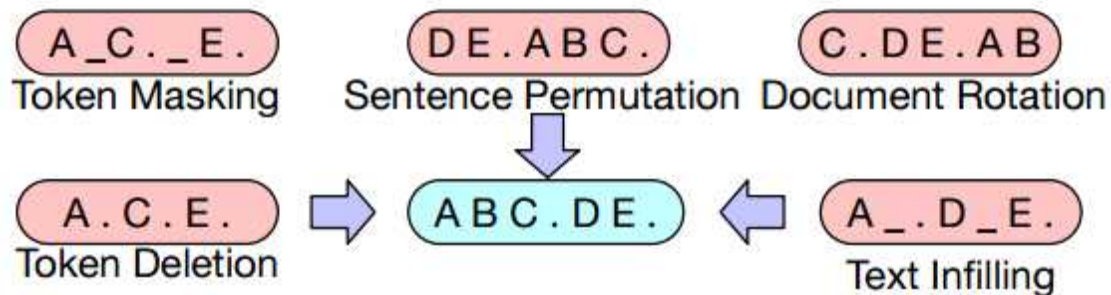
(BERT, GPT 제외)

- UniLM : **조건부 독립**(conditionally independent)의 형태로 예측을 하는데, 마스킹된 여러 토큰을 예측할 때, 각 예측이 서로에게 영향을 주지 않고 독립으로 이루어짐.
BART의 경우 다음 토큰을 예측할 때 이전에 자신이 생성했던 토큰들을 참고하므로, 실제 사람이 글을 쓸 때의 방식과 유사(더 자연스러움).

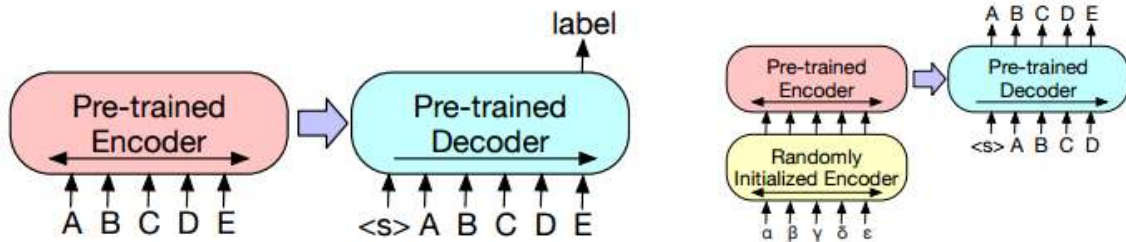
- MASS : 연속된 토큰 일부를 마스킹한 후, 인코더는 마스킹된 부분을 제외한 나머지 텍스트만 보고, 디코더는 마스킹된 부분만 예측하도록 학습하므로, 인코더가 손상된 텍스트 전체를 입력으로 받는 BART와 다르게 discriminative task에 덜 효과적임.

3. Method

- Corrupted document를 original document로 매핑하는 denoising autoencoder, negative log likelihood를 사용하여 사전학습.
- Architecture : 기본 seq2seq transformer 아키텍처 사용
GPT와 동일하게 ReLU 대신 GeLU 사용
base model은 encoder, decoder에 각각 6 layers, large는 12 layers
decoder의 각 layer는 encoder의 마지막 hidden layer와 cross attention을 진행(위에서 언급한 encoding)
BERT의 경우 mask 토큰을 맞추기 위해 어휘 전체의 확률 분포로 의미 벡터를 변환해야하는데, 이를 위해 Feed-Forward Network가 필요하나 BART의 경우 Decoder에서 단어가 예측되므로, 이미 내부에 있으므로 추가할 필요가 없음.



- Pre-training BART : 일반적으로 denoising autoencoder은 noise를 주는 방식이 정해져 있는데, BART는 corruption 모두 적용 가능
 - Token Masking : 랜덤 token을 mask 대체
 - Token Deletion : 랜덤 token을 input에서 삭제
 - Text Infilling : lambda가 3인 포아송 분포로 span length를 결정하고 text spans을 정한 span length만큼 샘플링하고 단일 mask token으로 대체
span에서 얼마나 많은 tokens이 빠져있는지 예측하도록 모델 학습
 - Sentence Permutation : 마침표 기준 문장을 나누고 순서 변경
 - Document Rotation : 특정 토큰으로 문서 시작점 변경



- Fine-tuning BART : Sequence Classification Tasks : encoder와 decoder에 동일한 input 삽입
Token Classification Tasks : 각 단어의 hidden state를 사용해서 토큰 분류
Sequence Generation Tasks : autoregressive decoder를 활용하여 fine-tune
Machine Translation : BART encoder의 embedding layer를 초기화된 encoder로 대체

4. Experiments

실험 구성

- SQuAD(질의응답), MNLI(자연어 추론), 요약, 대화 등 여러 task에서 비교
- Text Infilling + Sentence Shuffling 조합을 noise function으로 사용

실험결과

- Comprehension Task : 초기 대조군인 RoBERTa와 거의 대등한 성능을 보이며, 이는 BART의 디코더 구조가 문장 이해 능력이 밀리지 않음을 시사.
- Generation Task : SOTA 압도적인 능가

5. Discussion

발견한 점

- 양방향 인코더와 자동회귀 디코더를 결합한 seq2seq 사전 훈련 방식이 전반적으로 뛰어난 성능을 보임.
- 어떤 노이즈를 주느냐에 따라 task별 성능이 크게 달라졌는데, Text Infilling 방식이 전반적으로 안정적이고 높은 성능을 보임.

한계점

- 입출력의 연관성이 낮은 경우 BART의 능력이 낮아지는 것을 보임.
- 문법적으로 그럴듯하게 보이지만, 없는 내용을 생성(Hallucination 위험성)

6. Conclusion

- 손상된 문서를 원본으로 복원하도록 학습하는 새로운 사전 학습 방법을 제안했으며, 기존의 task 능력을 잃지 않으며 우수성을 입증함.

기존 연구 대비 차별점

- BERT + GPT의 결합: BERT의 양방향 문맥 이해 능력과 GPT의 자동회귀 생성 능력을 하나의 seq2seq 프레임워크 안에 융합
- 유연한 noise function : 기존 모델들은 특정 마스킹 방식에 국한되었으나, 텍스트 삭제, 채우기, 문장 섞기 등 다양하게 텍스트를 손상시켜 강력한 학습 유도.

핵심 아이디어 요약

- 좋은 거랑 좋은 거를 합치면 더 좋은 게 나온다.

[Paper Preview] An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

25기 분석
서영우

0. Abstract

- Sequence Modeling 분야에서 RNN 기반 모델들이 활용되었는데, 특정 과제에서 CNN이 더 뛰어난 성능을 보이기 시작
- 간단한 합성곱 아키텍처인 TCN이 LSTM과 같은 대표적인 순환 신경망보다 다양한 분야에서 더 우수한 성능을 보이고, 더 긴 메모리(effective)를 가짐.

1. Introduction

- Sequence data를 다루는 대표적인 방법으로는 RNN 기반 모델인 LSTM, GRU 등이 전통적으로 사용되어 왔는데, RNN은 hidden state를 통해 과거 정보를 요약해서 현재 예측에 활용하는 구조이기 때문.
- 기존 RNN이 가진 Vanishing Gradient 문제를 LSTM과 GRU 모델이 해결하면서 긴 시퀀스를 다룰 수 있게됨.
- 음성 합성, 기계 번역 등 몇몇 분야에서 CNN 기반 모델들이 RNN 기반 모델들을 능가하면서, 단순히 특정 분야에서만 뛰어난 우연인지 의문 제기.

[대표적인 CNN 기반 순차 모델]

- WaveNet(2016) -> 오디오 합성에서 RNN보다 뛰어난 성능
- ByteNet(2016), ConvS25(2017) -> 기계 번역에서 LSTM보다 우수
- Gated CNN(2017) -> 언어 모델링에서 기존 RNN보다 좋은 성능

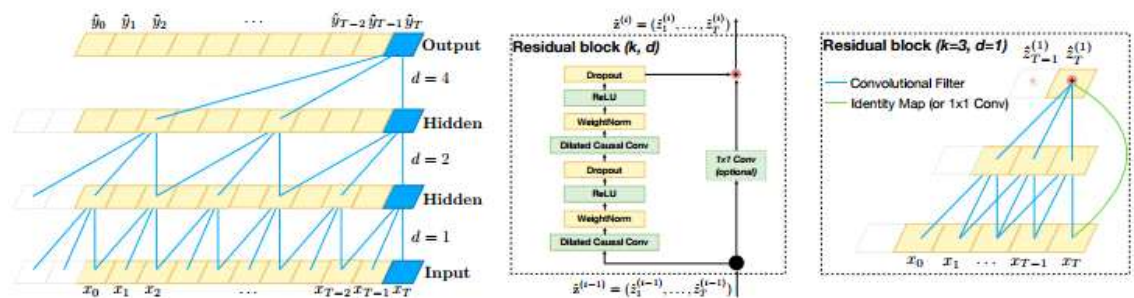
2. Related Works (Background)

- CNN의 경우 이미지 처리 위주로 활용되었으나, 음성이나 자연어 처리 등 sequence data 활용 목적으로도 사용됨.
- 그러나 LSTM 등장 이후 이를 능가하는 baseline을 찾기 어려울 정도로 강력하게 자리잡게 됨.
- 하지만 이는 RNN 기반 모델들과 비교를 했기 때문에 다른 패러다임과 비교하지 않았고, 이러한 배경이 본 논문의 시발점으로 작용.

3. Method

3.1 Temporal Convolutional Networks (TCN)

- 새로운 개념이라기보단, 기존 CNN 기법들을 시퀀스 모델링에 맞게 조합하고 단순화 진행



- 1) 인과적 합성곱(Causal Convolutions)

기존 CNN의 경우 입력 시퀀스를 처리할 때 양방향 정보를 활용하는데, sequence data의 특성상 미래 정보를 알고 있으면 안됨 (data leakage 위험성)

$$y_t = \sum_{i=0}^{k-1} w_i x_{t-i}$$

미래의 정보가 과거의 결과에 영향을 주지 않도록 설계. 시간 t에 대하여 예측할 때 t를 포함하여 그 이전(t-1, t-2, ...)의 입력만 사용. RNN의 autoregressive와 유사.

입력과 출력 시퀀스의 길이를 동일하게 유지하기 위해 1D Fully Convolutional Network 구조 사용. zero padding(0 대입) 추가 (앞부분에만 kernel size - 1)

- 2) 팽창된 합성곱(Dilated Convolutions)

기존 RNN의 경우 시간 순서대로 한 단계씩 정보가 전달되기 때문에, 과거 정보를 장기적으로 유지하기 어려웠고, LSTM, GRU로 넘어가면서 일부 개선되었지만, 여전히 장기 시퀀스를 다룰 때는 한계 존재.

$$F(s) = (\mathbf{x} *_{\mathbf{d}} f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i}$$

일반적인 합성곱은 네트워크 깊이에 비례하여 receptive field가 선형적으로 증가하기 때문에 팽창 계수 d를 사용해서 필터 내 입력값들 사이에 일정한 간격을 둬. d값을 지수적으로 증가시켜 적은 수의 layer로도 receptive field 확보

- 3) 잔차 연결(Residual Connections)

TCN의 네트워크가 깊어지는 경우 Vanishing Gradient 문제가 발생할 수 있음(학습X)

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x}))$$

두 개의 Dilated causal convolutional layers와 ReLU, Weight Normalization, Dropout으로 구성된 잔차블록을 통해 깊은 네트워크의 학습을 안정화 Resnet과 유사한 잔차 연결을 통해 기울기 소실 방지

- TCN의 경우 receptive field size를 여러 가지 방법을 변경이 가능하다(Dilation 크기, 레이어 크기 필터 크기 등)고 말하며 모델의 메모리 크기에 대한 유동적 제어가 가능하다고 언급
- But hyperparameter의 자유도가 높아지는 것이므로 best를 찾는 것이 어려워지지 않나 라는 **개인의견** (사전에 알 수 없으므로)

3.2 RNN Based Models

(1) RNN(Recurrent Neural Network)

- 과거 데이터를 hidden state로 요약하여 다음 시점으로 전달
- Vanishing Gradient의 문제로 긴 시퀀스를 학습하는 데 어려움

(2) LSTM (Long Short-Term Memory)

- Gates 구조를 통해 필요한 정보만 유지하는 방식으로 개선
- RNN의 기울기 소실을 해결했으나, 병렬 연산이 어려워 학습 속도가 느림

(3) GRU (Gated Recurrent Unit)

- 더 적은 매개변수를 사용하여 학습이 더 효율적임
- Long-term dependency learning에 한계를 가짐

4. Experiments

- 1) 장기 기억이 가능한지 판단하기 위한 Synthetic Stress Tests dataset
- 2) 순차적인 픽셀 데이터를 처리하는지 확인하기 위한 MNIST 및 변형된 P-MNIST
- 3) 음성 처리 능력을 판단하기 위한(Sequence data) JSB Chorales, Nottingham Dataset
- 4) 자연어 처리에 능한지 판단하기 위한 PennTreebank, WikiText-103, LAMBADA

위 데이터들을 통해 실험한 결과

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^{<i>h</i>})	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^{<i>ℓ</i>})	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^{<i>ℓ</i>})	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^{<i>ℓ</i>})	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

모든 실험에서 TCN이 RNN 계열 모델들을 능가했고, 특히 기억 유지력이 중요한 Copy memory나 Adding problem에서도 압도적으로 차이를 보였음.

MNIST 역시 더 높은 정확도를 기록

5. Discussion

- RNN과 달리 합성곱 연산은 병렬화가 가능하여 훈련속도가 빠르고, 역전파 경로가 시퀀스의 시간 순서와 다르므로 Gradient Vanishing 문제를 걱정할 필요가 없음.
- RNN의 경우 고정된 크기의 은닉 상태만 유지하면 되므로 과거 데이터를 직접 저장할 필요가 없으나, TCN의 경우 ConvNet 필터를 통해 처리하므로 예측을 위해 Receptive field만큼의 raw sequence를 메모리에 유지해야하는 문제점 존재.
- 메모리 효율성 면에서 그럴 수 있을지 몰라도 분산 처리가 가용하기 때문에 큰 문제가 되지 않는다고 판단(개인의견)

6. Conclusion

- 1) 장기 의존성 (Long-Term Dependency) 학습 유리
- 2) 병렬 연산 가능
- 3) 다양한 벤치마크 실험에서 RNN보다 우수한 성능을 보임

을 통해 Sequence data를 처리할 때 RNN이 무조건적인 선택지가 아님을 증명

기존 연구 대비 차별점

CNN 내부, RNN 내부가 아닌 CNN과 RNN의 패러다임 대결

코드 분석

- class Chomp1d(nn.Module) 부분에서 미래의 정보를 잘라내기 위해 chomp_size만큼 길이를 잘라냄.
- Conv1d -> Chomp1d -> ReLU -> Dropout 형식의 2개의 팽창된 인과적 합성곱 레이어
- downsample : Residual Connection에서 블록의 입력과 출력의 텐서 크기 맞추기 위함
- class TemporalConvNet(nn.Module) 부분에서
레이어가 깊어질수록 팽창 계수를 $dilation_size = 2 ** i$ 의 형태로 구성하여 긴 시퀀스를 효율적으로 학습 가능